

The Half-Ask

Stop Discounting the Incredible

Why organizations quietly shrink requirements before they are written, and where the cost actually lands.

IF YOU READ ONE PAGE, READ THIS ONE

The Short Version

Organizations often ask for less than they need. People quietly adjust requests to what they believe the team, the budget, the technology or the organization can deliver. The requirement is constrained before feasibility is ever discussed.

That is the half-ask. It looks complete, gets estimated, built and shipped. Metrics show success. What they cannot show is what was removed before the requirement was written. The missing ambition leaves no artifact and no backlog item.

Behind every half-ask sits a private price list: an internal model of what is realistic—what the team can build, how long things take, what will be funded, and how much political capital an ambitious ask will cost. Much of that model is rational. Some of it is stale, inherited from a time when development and experimentation were far more expensive.

There are two forms of the same discount. Need-discounting: we know what we want but ask for less because we predict it cannot be delivered. Discovery-discounting: we do not know what is possible and never investigate because finding out seems too costly. One shrinks a known need. The other leaves a possible future unexplored.

The economics have changed. AI-assisted development is cutting not only the cost of building but the cost of finding out. Ideas that once needed a funded project simply to investigate can now be prototyped or explored cheaply. Delivery is not free, but many old assumptions about what is worth asking are newly questionable.

The discount is not a saving. It is a transfer. What disappears from the development estimate often reappears as support, workarounds, rework, slower future delivery or lost opportunity. The team that took the discount may record a success while another part of the organization quietly pays the bill.

This is why clean delivery metrics can mislead. A backlog can be completed and velocity healthy while the organization systematically under-asks. Traditional measures show whether we delivered what we committed to. They are far worse at showing what we never allowed ourselves to request.

The proposal is simple: separate possibility from feasibility. First ask *what would genuinely solve this* — without silently pricing it against today's constraints. Then ask what we can deliver and by when. Make the gap visible rather than deleting it before anyone can see it.

The gap itself is valuable. If full possibility repeatedly exceeds what the organization can deliver, that difference reveals a capability gap, a funding decision or a strategic choice. Once visible, it can be owned.

This is not a case for ignoring constraints. Feasibility, security, budgets and capacity remain real. The argument is about sequence: apply those constraints after the possibility has been honestly described—not invisibly while deciding what is permissible to imagine.

Stop discounting the incredible before you know what the incredible costs.

The Half-Ask

A UX designer sits down to contribute to a requirement. After thorough user research, stakeholder interviews, and weeks of synthesis, he believes he knows exactly what the product needs. He also knows the team that will build it — he has read their skill composition, their history, their scar tissue — and he knows they cannot deliver it. So he contributes half of what he knows.

Nobody told him to do this. Nobody will ever know he did it. The requirement that reaches the backlog looks complete, reasonable, well-formed. It will be estimated, built, and shipped, and the delivery metrics will record a success. What the metrics cannot record is that the ask was discounted before it was ever voiced.

The designer is a composite. The behavior is not — it is drawn from many such moments across three decades of practice, including my own.

This paper is about that discount: where it comes from, why it is individually rational and collectively corrosive, and why the moment to correct it is now — because the economics that made the discount feel like realism have quietly moved. The tools that once made testing a ceiling expensive now make exploration cheap. The cost curve has inverted. Most private price lists have not.

The Private Price List

A requirement is never just a description of a need. It is a price quote, written by someone who has already estimated what the ask will cost — in delivery time, in political capital, in the risk of looking unreasonable — and adjusted the ask to a price they believe will be accepted.

The undiscounted version is the requirement at full price: the ask that reflects the actual need, or the actual ceiling of the problem, before anyone quietly shrank it to fit anticipated capacity. Almost no requirement is written at full price. Most are pre-discounted through a private, invisible act of judgment that the writer often does not notice performing. The designer in the opening did not experience himself as suppressing a need. He experienced himself as being realistic.

That is what makes this a habit of mind rather than a process flaw. Process flaws are visible in artifacts and can be audited. This discount happens upstream of every artifact, in the moment before the first word of the requirement is written.

This is anticipatory scope-limiting, and it is distinct from Conway's Law. The system does not merely mirror the organization's communication structure; the ask itself is pre-filtered by a private model of what the team can handle, often before the team is even in the room.

Not every organization discounts. There are environments — the space industry, motorsport, offshore yachting, haute couture, luxury automotive — where the half-ask was never viable. When the requirement is a lap time set by physics, a garment worn by one person on one night, or a system that must function in orbit, the gap between what is needed and what is asked closes to zero by necessity. The cost of the discount is immediate and visible. These industries did not develop the habit this paper describes because they could never afford to.

Naturally, a small number of prestige projects in other industries also operate at full price — certain flagship products, high-stakes platforms, or executive-sponsored initiatives where the cost of the half-ask is politically or commercially intolerable. If you work on one of these, you are the exception. Most organizations are not so fortunate. This paper is written for the far more common case: the environments where the discount remains invisible, the metrics stay clean, and the innovation that was never requested leaves no trace.

The habit is individually rational and collectively corrosive. Rational, because pushing an ask you expect to be rejected or half-built wastes credibility and political capital. Corrosive, because it creates a ratchet: the team is never stretched, leadership never sees the gap between what users need and what ships, and "we can't do X" becomes true by never being tested. Worse, the discount hides its own cost. An organization whose requirement writers always contribute half will show clean delivery data — reasonable asks, met on time — while a capability gap grows that no metric is tracking. The plans fill up, delivery objectives are met, and the performance metrics look healthy. What nobody sees is the innovation that never made it into the plan.

A Thought Experiment

Before the next planning session, try this.

You already know a little about what your team has available. You know which holidays fall inside the planning period. You know the planning cadence, roughly who is on leave, which two people are half-allocated to something else. You are probably factoring those in — just a little — as you decide what to put forward.

Now argue with yourself. Imagine you had a hundred developers available, in addition to the team you already have. Would that change what you asked for?

If the answer is yes, you have just found your private price list. The holidays were never the interesting part. The interesting part is everything else on that list you did not notice consulting.

The experiment is not hypothetical for much longer. Capacity at that scale — human, agentic, or both — already exists in organizations driving innovation today, and the cost of accessing it is falling faster than most planning habits are updating.

The Literature That Institutionalized the Discount

The requirement writer does not invent the discount alone. A substantial body of respected literature created the conditions in which it thrives.

Hersey and Blanchard's Situational Leadership — still one of the most widely taught leadership frameworks — says explicitly: match your style to the team's competence and confidence, not to the process you would prefer. Skelton and Pais's Team Topologies makes team cognitive load a first-class design constraint; work should be shaped to fit what the team can absorb. Mike Cohn's Agile Estimating and Planning institutionalizes planning by yesterday's velocity. The Scrum Guide asks Developers to consider past performance and upcoming capacity when forecasting what can be completed in a Sprint. SAFe's PI Planning explicitly matches demand to available capacity.

All of this is sound advice for delivery. None of it cleanly distinguishes between adapting the staging of a full-price requirement and quietly deleting the full-price requirement before anyone sees it. The literature was written for a world where capacity was expensive and slow to change, so adapting to it was wisdom. Over time the discount became indistinguishable from craft.

It goes further. Karl Wiegers and Joy Beatty's *Software Requirements* includes feasible among the characteristics of good requirements. A Guide to the Business Analysis Body of Knowledge (BABOK Guide) does too — while importantly allowing that a requirement can be considered feasible enough to investigate further through experiments or prototypes. The tension is not that these standards forbid ambition. It is that feasibility enters the quality conversation early enough to encourage premature discounting when applied too soon.

The requirements field always knew a version of the problem existed. Suzanne and James Robertson's *Mastering the Requirements Process* identifies "undreamed-of requirements" — needs that stakeholders never voice because they have not imagined them or do not realize they are possible. The Robertsons named it. Yet mainstream practice also evolved feasibility checks, readiness criteria and capacity-based planning that can, when applied too early, filter such requirements before they are explored.

Jeff Patton's *User Story Mapping* and Gojko Adzic's *Impact Mapping* push back — both argue for mapping the whole need first and slicing for delivery afterward. Marty Cagan's *Empowered* argues teams should be given problems to solve, not pre-shrunk feature lists. These are important counter-voices. On balance, however, the canon still treats early feasibility as a professional virtue rather than a risk to be managed.

Kahneman's *Thinking, Fast and Slow* supplies part of the underlying psychology. Several well-documented biases can push the ask downward, while professional practices developed under an earlier cost regime can reinforce them.

The Biases That Corrupt the Price

If the discount were a clean, current calculation it might be defensible. It is neither. The private price list is corrupted by predictable biases, and because the pricing happens invisibly, the corruption is never audited.

Anchoring on past delivery. New requirements are priced against how long the last similar thing took, even when tooling, staffing, or the problem itself has changed. The anchor is stale but feels like realism. Past delivery performance becomes the invisible ceiling on what gets asked for next.

Availability bias on failure. One visible failure of an ambitious ask colors requirements for years, while many quiet feasible wins leave no impression. The price list overweights the memorable disaster.

Status quo framing. Requirements are written as deltas from the current system rather than from the ideal outcome, so the cost being estimated is the cost of change, not the cost of the capability. Features get written; outcomes do not. A team never asks whether onboarding could go from twenty minutes to two, because everyone assumes that would mean a rewrite — and the rewrite, not the outcome, becomes the thing being priced.

The cost-of-asking tax. Every requirement carries a social cost — looking unreasonable, spending credibility with the product manager, appearing to underestimate complexity — that is separate from its delivery cost. Writers optimize against the social price, not the real one. This is the UX designer's half-ask, generalized.

Skill-composition proxy error. Feasibility is priced against who is on the team today, rather than what is learnable, hireable, or automatable within the timeframe.

The requirement-writer's curse. The person contributing to the requirement often feels responsible if it is not met, so they unconsciously de-risk the ask rather than record the true need. This bias deserves particular attention: the same person who sees the full need is also the one who will carry the political cost of an unmet ask. The incentive is structurally misaligned.

Premature solutioning. Requirements written in implementation-shaped language ("add a filter dropdown") rather than outcome-shaped language ("let users narrow results") lock in the old cost model before anyone reconsiders it.

Sunk-cost conservatism. "We built our workflow around constraint X" remains a reason to write X-shaped requirements long after the constraint is gone.

Every one of these biases pushes in the same direction: downward. The discount compounds.

Two Forms of the Same Discount

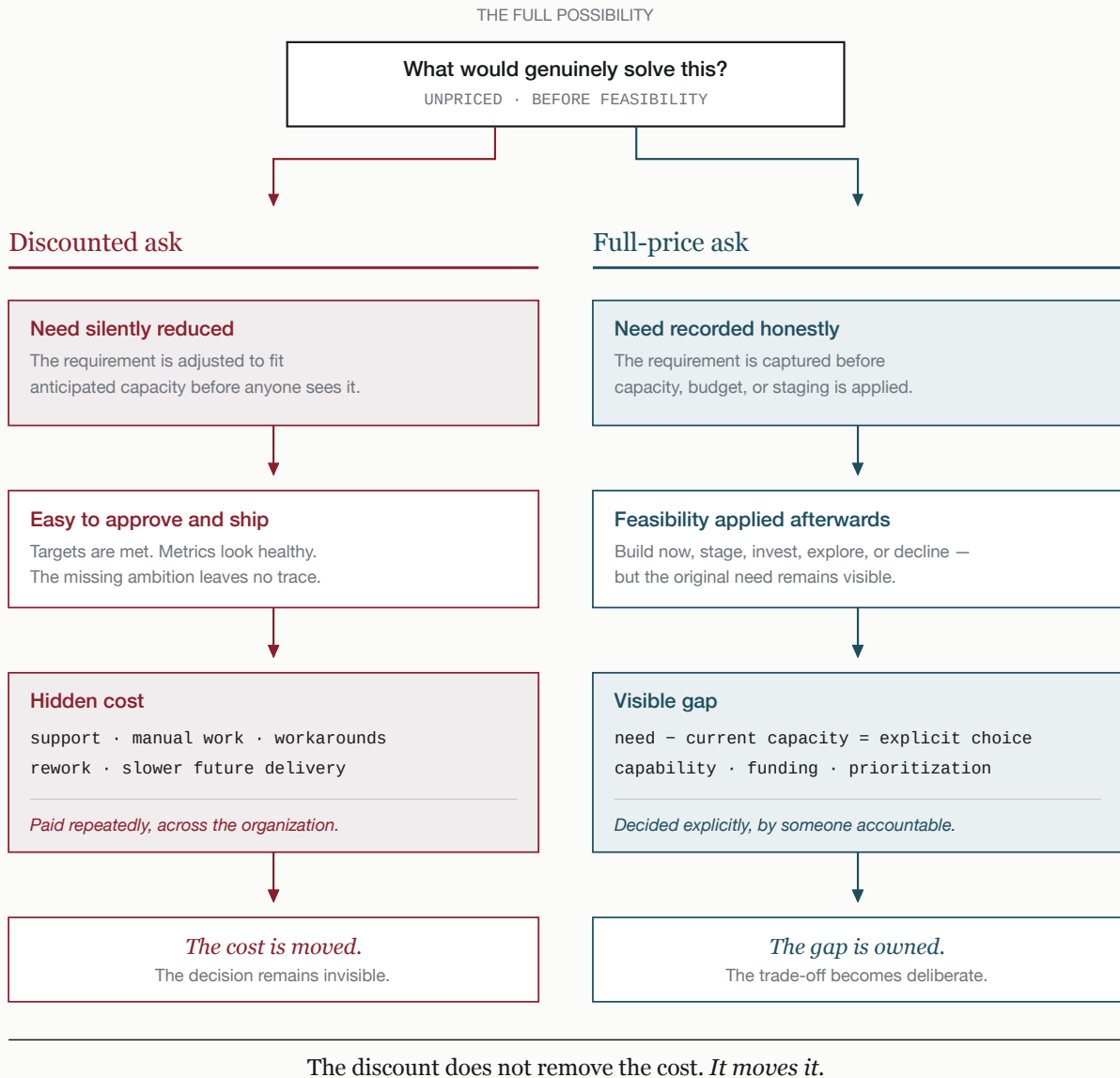
So far the discount has been described at the scale of one person and one team: a known need, quietly shrunk. The same habit of mind operates at larger scale, where it takes a more consequential form — not the suppression of a known need, but the failure to ever test what is possible.

Consider a product organization that has lived with a twenty-minute onboarding flow for six years. Nobody has ever asked whether it could be two minutes. Not because anyone measured the effort and rejected it, but because the question was never separable from the answer everyone assumed: a rewrite, a year, a team that does not exist. So the flow gets incrementally improved — nineteen minutes, eighteen — and the two-minute version stays permanently unpriced. The ceiling was never tested. It was inherited.

The same pattern operates at industry scale. Consider a transportation organization facing an aggressive target — say, cutting fuel consumption by ten percent. The question may never be tested, not because anyone calculated that ten percent is impossible, but because finding out whether it was possible was historically too expensive to attempt. Targets can default to the ceiling of past capacity rather than the ceiling of the actual problem. Over time those defaults can calcify into industry common sense. Nobody needs to discount a known need for an entire field to stop pricing the question.

FIGURE 1

The discount is not a saving. It only decides *where the cost appears*.



These are two forms of the same discount:

Need-discounting — I know what I want, and I contribute less because I predict the team cannot deliver it.

Discovery-discounting — I do not know what is possible, and I never find out, because the cost of finding out exceeded what the answer seemed worth.

Where the Discounts Actually Live

When a requirement is written at half price, the cost of the missing half does not evaporate. It moves — out of the development estimate, where it would have been visible, owned, and argued about, and into parts of the organization that never sat in the requirements conversation. The delivery team ships on time and on budget. Somebody else pays.

To customer support. The onboarding flow that could have been two minutes stays at twenty. Every new user now needs guiding through it, formally or by trial and error. Tickets accumulate around "how do I..." The support function absorbs the cost of what was never built. One requirement, discounted once, becomes years of support load — and nobody computes the total, because support headcount and feature estimates live in different budgets.

To operations. A workflow that could have been fully automated ships at half automation, requiring manual intervention at three decision points. Operations manages the exception handling, documents the manual steps, trains every new hire on them. The discount converted "build it right" into "build it and we will manage it" — permanently, in headcount.

To everyone's working day. This is the most expensive form and the least visible. A mandated internal system — time reporting, expense approval, access requests, anything the business rules require people to visit regularly — is built with the awkward parts left out, because trimming them looked like a sensible saving during development. Now it takes ninety seconds instead of thirty, or four clicks instead of one. Multiply by the whole organization, every week, for the life of the system. Thousands of hours a year, drawn from every function simultaneously, appearing on no line item and attributed to no decision. A system that everybody is required to use is the single worst place in an organization to take a discount, and it is routinely where the discount is taken first — precisely because the users are captive and cannot churn.

To every team downstream of a foundation. A design system is mandated across the portfolio and ships without full component coverage, without interaction states specified, without accessibility built in. It cannot be avoided, and it cannot do what teams need. So every team extends it, works around it, or quietly rebuilds it. The artifact intended to enforce consistency becomes the origin of fragmentation, and the accessibility debt is inherited by every product at once — so when it is finally caught, remediation is not one project but all of them. The discount was taken once, at the foundation. It is paid by every floor above it, indefinitely.

To retention. The requirement that solved sixty percent of the user's problem did not fail. It underperformed, quietly. Users adapt, work around it, or leave. The cost surfaces in churn, a metric nobody traces back to a requirement written three years earlier.

To future velocity. The MVP that was explicitly temporary becomes permanent, because revisiting it means deprioritizing something else. Every team that inherits it pays comprehension tax on the shortcuts and the undocumented decisions. A one-time saving in an estimate compounds into years of slower delivery elsewhere.

To compliance and security. A feature ships without full security modeling because it will be handled next cycle. Audit finds it. What could have been part of the original build becomes urgent, unplanned, expensive rework. The cost was not avoided; it was deferred and amplified.

The pattern is consistent. The cost of the discount is real, it is large, it is distributed, and it is structurally divorced from the decision that caused it. The team that took the discount records a success. The functions that absorb it record a burden with no identifiable origin. The organization pays the bill in full and never sees the invoice.

This is what makes the private price list more than an estimation error. It is a misallocation of accountability. And it means the honest comparison was never "expensive requirement versus cheap requirement." It was "cost paid once, visibly, by the people equipped to argue about it" versus "cost paid forever, invisibly, by everyone else."

Which reframes the diagnostic problem usefully. You cannot audit a private price list directly — it leaves no artifact. But you can audit its consequences. Support volume clustered around a single workflow, an operations team whose headcount is justified entirely by exception handling, a mandated internal tool everybody complains about, a design system with a shadow library of local overrides: these are not unrelated operational irritations. They are receipts.

The Cost Curve Flipped

The economics underneath both forms of the discount have moved. Most price lists have not been updated.

The obvious shift is delivery cost. AI-assisted development, design, and prototyping mean that asks which were previously not worth requesting are now viable. A single BA or product person can stand up a working internal tool in days rather than quarters. The more important shift is subtler: the new tools do not just lower the cost of delivery — they collapse the cost of finding out.

A question like "can we cut fuel consumption ten percent?" no longer necessarily requires a funded research program to test. What happens when hundreds of parallel agents work the problem for two weeks? Nobody knows — and that is precisely the point. The question may previously have been too expensive to ask. As exploration gets cheaper, many previously untestable ceilings become cheap enough to explore. In those domains, discovery-discounting loses much of its rational justification.

The same inversion is visible at the tooling level. For two decades, "buy before build" was the rational default. That asymmetry has collapsed. Off-the-shelf platforms have grown more complex and expensive precisely because they try to serve everyone; organizations pay the license plus the ongoing configuration, plugin, and integration tax of bending a generic tool into their specific workflow. The question is no longer build versus buy; it is who pays the customization tax — once, at build time, or forever, in licensing and workaround time. A default assumption about cost, held confidently for twenty years, has inverted. Most

requirement writers are still pricing against the old curve.

One honest caveat is required. Cheap exploration creates its own failure mode: AI-assisted output that outpaces the team's understanding of it — comprehension debt in place of vendor lock-in. Validation, security, compliance, and integration costs do not disappear; in some cases they rise. Generation is cheap; verification is the new bottleneck. The new cost curve is not free. The trade is

real and must be managed deliberately. It is still a new trade, and it should be priced as one — not avoided by keeping the old price list.

Restoring Unpriced Discovery

The biases share a common structure: they import the answer to "what is feasible?" into the question "what is needed?" before the question is even asked. The correction is not to argue against each bias individually. The correction is to structurally separate the two questions, and to protect the space between them — the space where unfiltered need lives.

Call that space unpriced discovery: the part of the process where the requirement is described before it is costed. Not because constraints are unreal, but because pricing too early is precisely the mechanism of the discount. Unpriced discovery is not naivety. It is a disciplined deferral of the feasibility conversation to the people and the moment where it can actually be acted on. Nobody has forbidden the requirement writer from asking for more; the discipline is to notice the price list and set it down for one conversation.

This is not a call for unconstrained feature creep. It is a call to distinguish destructive discounting — suppressing genuine requirements because of fear, stale assumptions, or outdated cost models — from constructive staging — acknowledging real constraints while keeping the original intent uncorrupted and visible. The goal is that the gap between need and capacity becomes someone's deliberate decision rather than nobody's private habit.

Separate the conversation, not the people. Hold two distinct conversations, never one. The first asks only "what would genuinely solve this?" The second asks "what can we deliver, and by when?" Unpriced discovery lives in the first; feasibility lives in the second. This is a separation of mindsets, not of participants — engineers and architects belong in the first conversation, and their technical insight is often what reveals the real ceiling. What does not belong is the estimation reflex: no velocity talk, no "we tried that," no "the team can't." Those are second-conversation moves, and they need explicit facilitation to keep out of the first.

Manage the expectation gap deliberately. If the first conversation routinely produces asks that the second can fund at only twenty percent, contributor fatigue is inevitable. The remedy is not to abandon the separation; it is to make the gap itself a first-class object of attention. Record the full-price requirement, record the staged

version, and treat the difference as an explicit organizational signal — a capability gap, a funding gap, or a prioritization decision. Over time, patterns in that gap become the most honest diagnostic the organization has.

Make the undreamed-of requirement a deliverable. The Robertsons named this category decades ago; the profession immediately built gates to filter it out. Reverse that. Make at least one undreamed-of requirement — an ask that assumes current constraints do not apply — a required output of every discovery phase. Not because it will be built, but because the act of writing it recalibrates the contributor's imagination and puts the untested ceiling on record. Some of those

ceilings will later be tested and found lower than assumed. That is the organizational learning the discount was hiding.

Price the exploration separately. "Find out whether X is possible" was previously bundled into the cost of "deliver X," making the question unaskable without committing to the answer. Separate them. "What would we learn if we let agents work this problem for two weeks?" is now a budgetable line. Treat ceiling-testing as its own lightweight project type, distinct from delivery.

Audit the price list explicitly. When a feasibility judgment surfaces — "that would take months," "this team can't do that," "we tried something similar last planning cycle" — treat it as a hypothesis, not a fact. Ask two questions: when was this last tested, and has the cost of testing it changed? Most price lists are years stale.

Use creative constraint as a forcing function. "Solve this as if cost were not a factor." "Design this for an organization ten times the size." "Assume the technology you need already exists — what does the requirement look like?" These prompts separate the requirement from the delivery context long enough to see what the requirement actually is. The resulting full-price requirement may then be staged, phased, or deferred — but it exists, and it is on record.

Create organizational permission explicitly. Self-censoring is cultural as well as cognitive. A contributor who has had ambitious asks returned as unrealistic learns not to make them. Protecting unpriced discovery requires an explicit signal: that writing a full-price requirement will not be read as naivety, that the gap between it and the delivered version will not be held against the writer, and that the undreamed-of requirement is a contribution rather than an embarrassment.

Together these practices do not eliminate the feasibility conversation. They relocate it downstream of the requirement, upstream of the delivery plan, and into the hands of people with the authority to change the constraints rather than only work within them. Unpriced discovery is not the abolition of realism. It is the insistence that realism arrives after the need is honestly named, not before.

Addressing the Obvious Objections

Skeptical readers will raise three predictable concerns. Each deserves a direct answer.

"Writing the full-price requirement commits us to building it." No. The full-price requirement is not a commitment to build. It is a record of the true need and a calibrated input to prioritization. The time cost of writing one undreamed-of requirement per discovery cycle is measured in hours, not delivery cycles. The cost of never seeing the gap is measured in years of invisible under-ambition.

"Large gaps between the two conversations will demotivate people or damage psychological safety." It will if the gap between the two conversations is left unmanaged. The practices above treat the gap as visible and deliberate. Teams are not asked to deliver the impossible; they are asked to help make the trade-off explicit. Psychological safety is protected by the organizational signal that full-price writing is valued, not punished.

"But early feasibility checking is good professional practice." Early feasibility is valuable — when it is applied at the right moment, by the right people, against current rather than stale data. Importing it into the needs conversation itself is what this paper challenges. The distinction is timing and ownership, not the legitimacy of constraints.

The Habit That Outlived Its Rationale

The designer who contributes half is not failing at his job — he is doing it well by the standards of a reality that is being dismantled. Reading the team, adapting the artifact, pricing the ask against reality: these remain the disciplines of senior practice. What has changed is what reality costs.

The professional canon that codified the discount was built for a world where capacity was genuinely expensive to change, exploration was genuinely expensive to run, and feasibility was a reasonable quality gate because the cost of infeasibility was high. That world has moved. The exploration that once required a funded program may now require weeks rather than months. A tool that once required a dedicated engineering team may now be testable by a practitioner in days. Ceilings that entire industries stopped testing may now be cheap enough to revisit.

The discount that once passed for realism is now a stale price list, applied by habit, hiding a widening gap between what organizations ask for and what has quietly become possible. Every undreamed-of requirement that stays unwritten, every ceiling that goes untested, every half-ask that reaches the backlog as if it were the honest need — these are the compounding cost of a habit of mind that outlived its rationale.

Stop discounting the incredible.

Now Go Chase It

You have the mechanism now. The private price list, the biases that corrupt it, the canon that graded the corruption as craft, the two forms the discount takes, and the places the cost quietly lands. You also have the thought experiment, which is the only part of this paper that works on you rather than on your organization.

So the useful thing to say at the end is not another exhortation. It is that once you can see the habit, you cannot easily unsee it — and that is the whole point. You will start noticing it. In the requirement marked down before it reached the backlog. In the "we don't have capacity" stated as fact by someone quoting a test nobody has run since 2023. In the mandated tool everyone works around. In the initiative nobody proposed because the exploration spend felt unjustifiable against a ceiling nobody has priced.

Go chase the suboptimal.

Not because your organization is failing. It is probably excellent — at execution, at quality, at moving quickly inside its constraints. That is exactly why the discount survived: it never looked like a defect, it looked like competence. But the constraints have moved, and the price list has not.

Start narrow. One conversation held in two halves. One requirement recorded at full price alongside the version you can actually fund. One ceiling tested instead of inherited. One support queue traced back to the decision that created it.

You know where the discounts live now. Go and collect.

About This Paper

Disagreement is the point. If part of this is wrong — and some of it will be — the author would rather hear it than not.

Cite as Karlsson, J. (2026). The Half-Ask: Stop Discounting the Incredible. Otrobonita AI Labs. otrobonita.com/whitepapers. DOI: 10.5281/zenodo.21703599

License Creative Commons Attribution 4.0 (CC BY 4.0). Share it, quote it, argue with it — attribution is all that is asked.

The authoring team

- **Jesper Karlsson** — concept, thesis, ideas, structure, and accountability for every final call.
- **Fable 5, GPT 4.5, Grok 4.5, and Gemini 3.6** — drafting support, fact-checking, handling of uncertainty and counter-evidence, independent critique, and adversarial review.

Five iterations and approximately twenty-two individual feedback loops.

Written in 2026.

Hopefully useless by 2028.

The half-ask exists because the old cost curve made it rational.

That curve has already flipped.

If organizations stop discounting the incredible now, this paper should age quickly.